

Lattice Visualization Gallery

Daniel Ari Friedman

ORCID: 0000-0001-6232-9096

Email: daniel@activeinference.institute

DOI: 10.5281/zenodo.16887791

October 07, 2026

Contents

1 Lattice Visualization Gallery	1
1.1 Overview	1
1.2 Frequency shells	1
1.3 Lattice-plane slices	1
1.4 Dynamics strips	2
1.5 Deterministic animations and gallery plots	2
1.6 Reproducibility and test contract	6
1.7 Cross-references	6

1 Lattice Visualization Gallery

1.1 Overview

This section is the figure surface of the lattice layer: every rendering primitive used by the IVM chapters lives in `vis_lattice.py` (`shell_scatter`, `field_slice`, `dynamics_strip`, `gallery`). Each primitive receives its matplotlib axes explicitly, so panels compose inside caller-owned figures; only `gallery` creates figures (three, written as PNGs). Nothing renders at import time, and all randomness is drawn from a seeded `numpy.random.default_rng`, so the gallery is deterministic. The command-line entry is `quadmath/scripts/lattice_gallery.py` — a thin orchestrator that sets a headless backend and a fixed seed, delegates to `gallery` (thin-orchestrator contract of `quadmath/scripts/AGENTS.md`), and prints each written path on its own line: those stdout lines are the `make_all_figures` manifest contract.

1.2 Frequency shells

`shell_scatter(ax, sites, k)` embeds the sites of shell k (from `:func:omni_numbering.generate_shell`, which enumerates the $10k^2 + 2$ sites of shell k ; see [Lattice Tooling](#)) with `to_xyz` under `DEFAULT_EMBEDDING` and scatters them in 3D. With `axis_hints` it overlays four dashed rays from the origin toward the embedding matrix columns — the images of the quadray basis directions A, B, C, D , which are the vertices of a regular tetrahedron (pairwise dot product -1 , edge length $2\sqrt{2}$; see [Quadray Methods](#)). The hints make the tetrahedral axis frame of the lattice readable in print.

1.3 Lattice-plane slices

`field_slice(ax, field, sites, plane)` renders a scalar `ivm_field.IVMField` as a heatmap restricted to a lattice plane. A plane is parametrized by an origin q_0 and two independent integer translation vectors u, v — by default two of the twelve IVM neighbor moves, both permutations of $(2, 1, 1, 0)$ — and its points are

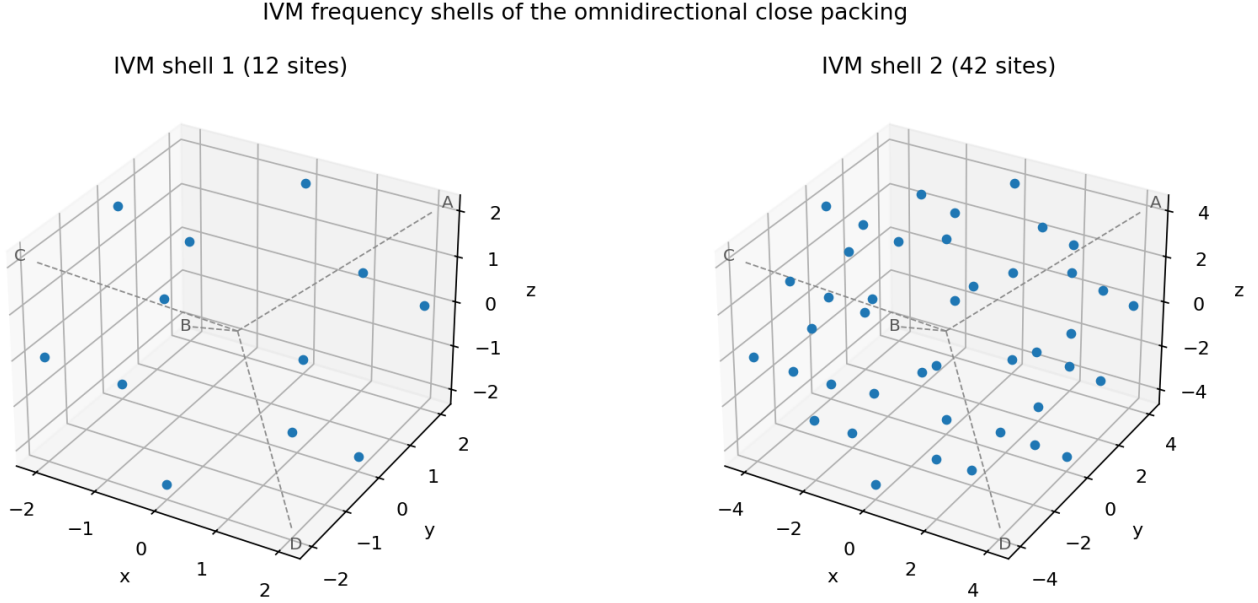


Figure 1: **Frequency shells of the omnidirectional close packing.** Shell 1 (12 sites) and shell 2 (42 sites) of the IVM lattice embedded via `DEFAULT_EMBEDDING` and scattered by `shell_scatter` as 3D point clouds in the x, y, z embedding coordinates; dashed gray rays mark the four tetrahedral quadray axes, labeled A, B, C, D, pointing toward the embedding-matrix columns. Reproduced with `uv run python quadmath/scripts/lattice_gallery.py` (fixed seed 12).

$$q(i, j) = \text{normalize}(q_0 + i u + j v), \quad (i, j) \in \mathbb{Z}^2, \quad (1)$$

with `normalize` the quadray canonical representative. Because each neighbor move has component sum 4, the sum stays divisible by 4 along the whole plane, so every $q(i, j)$ is again a lattice site. Two coordinate facts make the rendering exact: the embedding is linear, so the xyz image is the planar set $t(q_0) + i t(u) + j t(v)$; and the shift $(1, 1, 1, 1)$ lies in the embedding kernel, so plane membership is tested in integer quadray space — a site lies on the plane of (1) iff $q - q_0 = i u + j v + m(1, 1, 1, 1)$ for integers i, j, m . `field_slice` decides membership by an exact least-squares solve followed by an integer verification, so no site is ever misassigned by floating point. Sites off the plane are ignored, plane cells outside the field’s ball are masked, and the heatmap axes are the integer plane indices i (steps along u) and j (steps along v).

The gallery figure learns a field on the radius-3 ball (147 sites) from noisy observations of a linear synthetic truth (25% of sites observed, noise level 0.1, regularization `lam=0.01`, kernel width 0.5) and slices it through the default plane $((2, 1, 1, 0), (1, 2, 1, 0))$; the learned surface follows the planar lattice, with masked tiles where the ball ends. Field learning itself is the subject of [Static IVM Field Learning](#).

1.4 Dynamics strips

`dynamics_strip(axes, trajectory, t_indices)` renders selected snapshots of an `ivm_dynamics.simulate` trajectory (see [Dynamics and Learning on the IVM Lattice](#)) as a strip of 3D scatter panels over the embedded lattice. All panels share one symmetric color scale $[-v_{\max}, v_{\max}]$ with $v_{\max} = \max_t |u_t|$ over the selected snapshots, so evolution is directly comparable across panels. The gallery strip shows heat diffusion ($\alpha = 0.45$, horizon $T = 20$) on the radius-3 lattice (13 sites) at $t = 0, 10, 20$: the seeded random initial field relaxes toward its lattice average — the visually flat mid panel and right panel are the L_2 -monotone decay of the heat lemma in action.

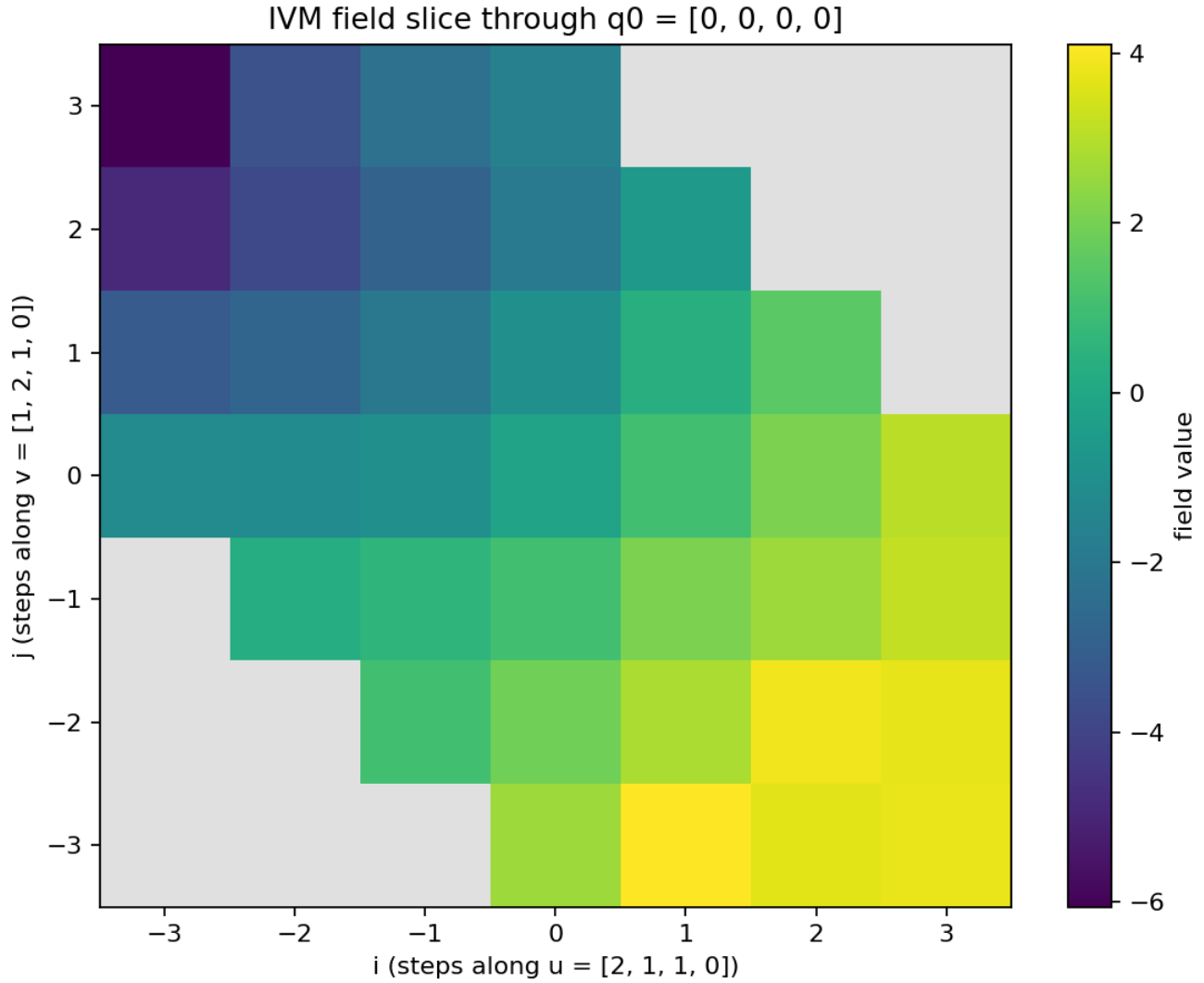


Figure 2: **Learned scalar field sliced along a lattice plane.** Heatmap of a Laplacian-regularized `IVMField` over the radius-3 ball, restricted by `field_slice` to the plane spanned by the neighbor moves $u = (2,1,1,0)$ and $v = (1,2,1,0)$: a viridis heatmap whose axes are the integer plane indices i (steps along u) and j (steps along v), with the colorbar reporting the field value and light-gray masked tiles marking plane cells outside the ball. Reproduced with `uv run python quadmath/scripts/lattice_gallery.py` (fixed seed 12).

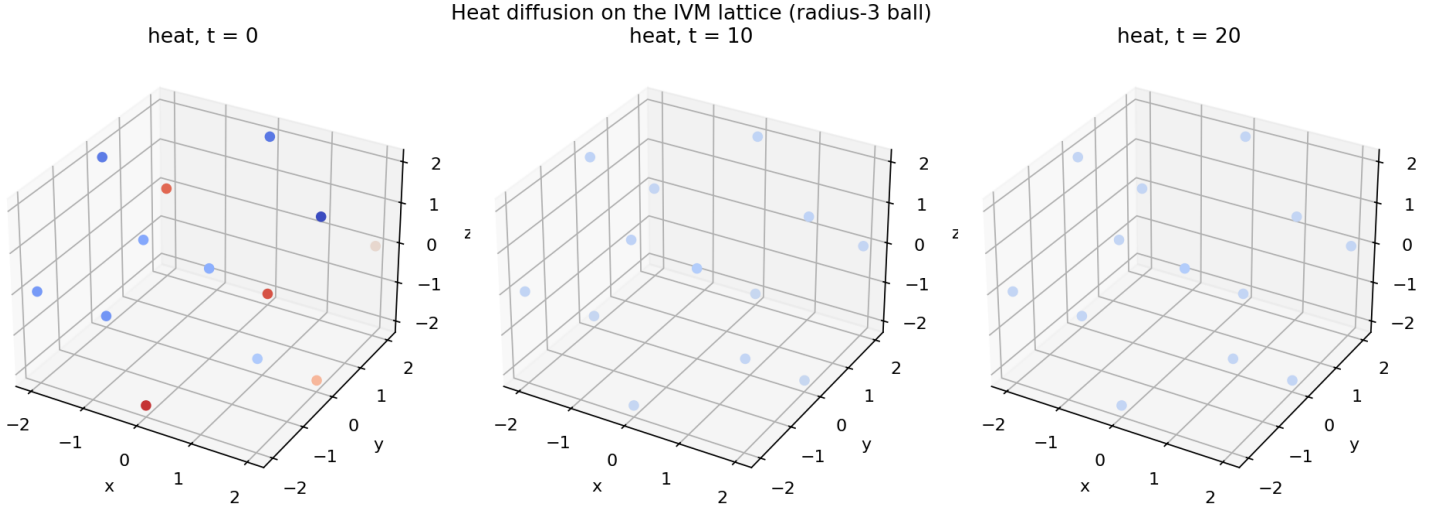


Figure 3: **Heat-diffusion evolution strip.** Snapshots of a seeded heat run at $t = 0, 10, 20$ on the radius-3 IVM lattice (13 sites), rendered by `dynamics_strip` as 3D scatter panels on one shared symmetric `coolwarm` scale $[-v_{\max}, v_{\max}]$ with $v_{\max} = \max_t |u_t|$ over the selected snapshots; the field visibly relaxes as the sum of squares decays monotonically. Reproduced with `uv run python quadmath/scripts/lattice_gallery.py` (fixed seed 12).

1.5 Deterministic animations and gallery plots

Where the primitives above compose panels inside caller-owned figures, the module `src/quadmath/viz/animations.py` renders frame-based animations that own their pixels: each renderer returns a list of `Frame` objects — the frozen `dataclass` `Frame` in that file pairs a 2D `numpy` array with a human-readable `title`, and its `__post_init__` validation admits only `uint8` arrays (raw gray levels) or float arrays with every value in $[0, 1]$, rejecting anything that is not 2D, is of another dtype, or drifts outside the unit interval. No `matplotlib` is involved at frame level and no RNG or wall-clock input exists (except the explicit seed of `diffusion_frames`), so identical arguments yield byte-identical frames. All three renderers share one fixed-camera convention: sites are embedded in R^3 via `DEFAULT_EMBEDDING` and viewed by an orthographic camera looking down $+z$ — each point (x, y, z) projects to (x, y) on a `GRID_SIZE = 48` square grid whose extent is $[-8, 8]$ (module constant `_HALF_EXTENT`, sized to cover a radius-3 IVM ball of embedded norm 6 with margin even while pulsing), the vertical axis flipped so $+y$ points up, colliding points resolved by maximum brightness, and out-of-window points clamped to the border pixel.

`simplex_frames(q0, q1, n=16)` animates a quaternion-slerp rotation of the radius-1 ball: the two unit quaternions q_0, q_1 (validated to four components of unit norm) are interpolated on the shortest arc — sign-flipping q_1 when their dot product is negative, using the Shoemaker spherical formula $q(t) = \sin((1-t)\theta) q_0 / \sin \theta + \sin(t\theta) q_1 / \sin \theta$, and falling back to normalized linear interpolation when the inputs are nearly parallel — and at each $t = i/(n-1)$ the 13 lattice sites of `ball_sites` radius 1 are rotated by $q(t)$, projected onto the 48×48 grid, and shaded by radial shell, with origin brightness 1.0 and the outermost shell never dimmer than 0.25 so nothing renders invisible.

`lattice_frames(shells=3, n=12)` renders a pulsing IVM lattice ball: the radius-shells ball from `ball_sites` (147 sites at the default radius 3) has its embedded coordinates scaled per frame by the deterministic pulse $1 + 0.25 \sin(2\pi i/n)$ over one full growth-and-contraction cycle, while brightness per site stays the fixed radial-shell shading — brightest at the center, decaying linearly in shell index, outermost shell at 0.25 — so the pulse animates geometry, not shading.

`diffusion_frames(n_steps=12, seed=0)` seeds one-hot heat on the radius-3 ball and diffuses it over the IVM neighbor graph: the 147 ball sites become a graph in which two sites are adjacent iff their difference normalizes to one of the twelve `IVM_NEIGHBOR_STEPS`, a `numpy.random.default_rng(seed)` draw selects the source site deterministically, and each explicit update applies $u \leftarrow u + \alpha$ (mean of graph neighbors $- u$) with $\alpha = 0.25$ (stable below 0.5) and values clipped at zero. Each frame is normalized by dividing by its (always positive) maximum heat, giving float values in $[0, 1]$; because heat only spreads to graph neighbors, the set of heated sites grows monotonically

across frames.

The companion module `src/quadmath/viz/plots.py` supplies standalone figure builders for diagnostics that do not belong to any gallery: `plot_loss_history` draws a training loss sequence as a marker line over its iteration index, `plot_shell_growth` plots the IVM shell cardinalities — the cuboctahedral numbers $10k^2 + 2$ produced by `shell_cardinalities` in `src/quadmath/lattice/ivm_field.py` — versus shell index, `plot_error_histogram` histograms error values with a dashed vertical mean line, and `plot_lattice_shell_3d` scatters the sites of one shell (`shell_sites`, embedded via `DEFAULT_EMBEDDING`) in 3D with equal-aspect axes. Unlike the axes-composing primitives of `vis_lattice`, each owns a fixed-size figure (6.4×4.8 inches at 160 dpi), applies a fixed style, and on `save=True` writes its PNG into `quadmath/output/figures/` via `quadmath.paths.get_figure_dir`, returning the output path (empty string when the caller keeps the figure). All four are input-agnostic and seed-free: inputs are plain sequences or shell indices, styling is fixed, and no randomness or wall-clock time enters, so runs stay deterministic headless (`MPLBACKEND=Agg`).

`frames_to_gif(frames, out_path, fps=8, scale=8)` assembles a frame sequence into an animated GIF: float frames are quantized to `uint8` gray levels by rounding $255v$, each frame is upsampled by the integer factor `scale` with nearest-neighbor resampling, and the sequence is saved with per-frame duration $1000 / \text{fps}$ milliseconds and infinite looping. PIL is imported lazily inside the function (Pillow stays optional at import time), and because the PIL GIF encoder is deterministic and embeds no timestamps, identical `(frames, fps, scale)` inputs render byte-identical GIF files — the same byte-stability contract the PNG gallery asserts.

`frames_strip(frames, out_path=None, labels=None, save=True)` renders the print still-counterpart of `frames_to_gif`: a single-row matplotlib strip with one grayscale `imshow` panel per frame, figure width scaling with the panel count while the height stays one panel plus the title band, each panel pinned to the unit brightness interval (`vmin = 0, vmax = 1`; `uint8` frames are first mapped by $v/255$, so both `Frame` dtypes share the GIF’s gray-level convention), per-panel titles taken from `labels` or, by default, from each frame’s own title, and axes switched off with zero inter-panel spacing so consecutive panels butt together. Matplotlib is imported lazily inside the function (the module stays matplotlib-free at import time), and saving follows the `plots.py` convention: a bare `out_path` name is written into `quadmath/output/figures/` via `quadmath.paths.get_figure_dir`, a path carrying a directory component is used verbatim, and the returned string is the written path ("" when `save` is off or no name is given). An empty frames sequence or a `labels` length mismatch raises `ValueError`; the render has no RNG and no wall-clock input, so identical inputs give byte-identical PNGs — the test suite asserts the byte stability, the full $[0, 1]$ grayscale excursion, the `uint8/float` equivalence, and the panel-count scaling.

The command-line surface is `quadmath/scripts/animation_gallery.py` — a thin orchestrator in the `quadmath/scripts/AGENTS.md` contract that sets the headless `Agg` backend and renders three GIFs via `frames_to_gif`: `animation_lattice.gif` (pulsing radius-3 ball, 12 frames), `animation_simplex.gif` (slerp from the identity to a 90-degree rotation about the x axis, 16 frames), and `animation_diffusion.gif` (12 diffusion steps from seed 0), writing each into `quadmath/output/figures/` and printing each path on its own line. The script is registered in `quadmath/scripts/make_all_figures.py` alongside the other galleries, so the manifest contract of the Overview applies unchanged: only path lines on `stdout`, byte-identical re-renders for identical inputs. The companion `quadmath/scripts/animation_stills.py` follows the same thin-orchestrator contract for print stills: it renders the three 6-frame sequences of the module — `lattice_frames` at radius 2, `simplex_frames` from the identity to a 90-degree rotation about z with the endpoint quaternion derived from `rotate_about_axis`, and `diffusion_frames` at seed 0 — samples each at an evenly spaced fraction (quarter-pulse, slerp midpoint region, final diffusion step), composes the three stills with `frames_strip` into `animation_frames_strip.png`, and prints the written path on its own line.

1.6 Reproducibility and test contract

- `gallery(paths_out_dir, seed=12)` writes exactly the three files `vis_gallery_shell.png`, `vis_gallery_field.png`, `vis_gallery_dynamics.png` (module constant `GALLERY_FILES`, in that order) into `paths_out_dir`, creating it when missing, and returns their paths. Every panel is fully seeded, and the PNGs carry no timestamp metadata: the test suite asserts byte-identical re-renders for a fixed seed.
- Tests: `tests/test_vis_lattice.py` — 19 tests, no mocks, deterministic assertions without pixel diffs (artist placement on caller-provided axes, exact field values at known lattice cells, byte-level reproducibility). Scope with `PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 uv run coverage run -m pytest tests/test_vis_lattice.py -q` and `uv run coverage report` — 100% statement and branch coverage of `src/quadmath/viz/vis_lattice.py`.
- Tests: `tests/unit/viz/test_animations.py` covers the frame module (Frame validation, the three renderers,

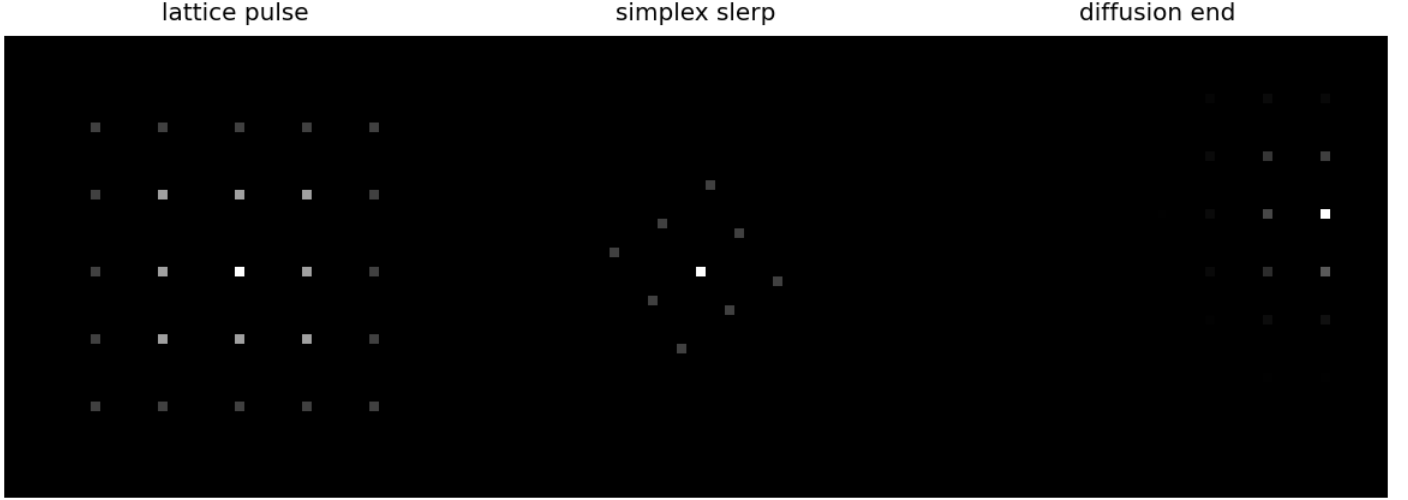


Figure 4: **Three animation stills in one strip.** Single-row `frames_strip` composition of one evenly spaced still from each 6-frame animation: left, the pulsing radius-2 IVM ball sampled at the quarter-pulse fraction (`lattice_frames`, radial-shell brightness, brightest at the center); center, the quaternion-slerp rotation of the radius-1 ball in the midpoint region of the arc ($t = 0.4$ on the way from the identity to a 90-degree rotation about z , `simplex_frames`); right, the final state of seeded heat diffusion on the radius-3 ball (`diffusion_frames`, brightness normalized by the frame’s maximum heat). All panels are 48×48 orthographic projections under the fixed $+z$ camera with brightness in $[0, 1]$; reproduced with `uv run python quadmath/scripts/animation_stills.py`.

`frames_to_gif`) and the strip renderer — `frames_strip` label and emptiness validation, byte-identical renders in temporary directories, `uint8/float` equivalence, the full grayscale excursion, and panel-count scaling — headless via `MPLBACKEND=Agg` (set in `tests/conftest.py`).

- Figure regeneration: `uv run python quadmath/scripts/lattice_gallery.py` (the script sets `MPLBACKEND=Agg` itself); stdout is exactly the three written paths.
- Still-strip regeneration: `uv run python quadmath/scripts/animation_stills.py` (the script sets `MPLBACKEND=Agg` itself); stdout is exactly the one written strip path, `quadmath/output/figures/animation_frames_strip.png`.
- Markdown: `uv run python quadmath/scripts/validate_markdown.py`.

1.7 Cross-references

- Site geometry, normalization, and the embedding: [Quadray Methods](#).
- Shell enumeration and nearest-site search: [Lattice Tooling](#).
- The sliced learner: [Static IVM Field Learning](#).
- The stripped dynamics: [Dynamics and Learning on the IVM Lattice](#).